

 MCA 2ND YEAR CLASSROOM LECTURE

MINING TECHNIQUES

Classification & Clustering in Practice

INSTITUTION

Moodalakatte Institute of Technology (MITK)

Kandavara, Kundapura, Karnataka

INSTRUCTOR

Abdul Muqeeet Armar

M.Tech CSE

INSTRUCTOR NOTE

Welcome MCA students to this 2-hour practical session. State clearly that today we demystify theory through code that matches their laboratory repository directly.

What Will We Learn Today?

01. Classification (Supervised)

1. Decision Tree

Hierarchical questions and if-else logic

2. k-Nearest Neighbors (k-NN)

Distance calculation and majority voting

3. Naive Bayes

Probability and word independence

02. Clustering (Unsupervised)

1. k-Means Clustering

Centroid recomputation & partitioning

2. Hierarchical Clustering

Agglomerative merging & dendrograms

From Concept → Algorithm → Python Code

INSTRUCTOR NOTE

Emphasize that we are strictly focusing on 5 core algorithms that match their lab folders: 3 for classification, 2 for clustering. No fluff, pure foundational clarity.

What is Data Mining?

“Data Mining is the process of finding **useful patterns** and **actionable knowledge** from raw data.”



10,000+ Student Records

Marks, attendance, study hours, lab performance



Analyze Data

Clean, filter, and run mining algorithms



Discover Patterns

“Attendance < 75% often correlates with backlogs”



Useful Knowledge

Timely remedial tutoring intervention

INSTRUCTOR NOTE

Analogy: Just as gold miners sift tons of earth to find valuable gold nuggets, computer scientists sift through gigabytes of raw data to discover valuable insights.

What are Mining Techniques?

Mining techniques are algorithmic methods used to discover structures, predict outcomes, or partition observations automatically. Today we master the two most important pillars:

Classification

Supervised Learning

Goal: Predict a known, predefined category or label.

The model is trained with past historical data that already contains correct answers (labels). It learns the mapping rule to predict the label of fresh, unseen data.

✓ Example: Will student **Pass** or **Fail**?

Clustering

Unsupervised Learning

Goal: Discover natural, hidden groups in unlabeled data.

No correct answers or class labels are given beforehand. The algorithm looks at similarities and distances between data points to group them automatically.

📍 Example: Grouping customers by purchasing habits.

INSTRUCTOR NOTE

Ask a student: "If I give you 100 emails and tell you which ones are spam, what technique is it?" (Classification). "If I give you 100 books without genre tags and ask you to group similar ones?" (Clustering).

Classification vs. Clustering

Feature	 Classification	 Clustering
Learning Paradigm	Supervised Learning (Teacher present)	Unsupervised Learning (Self-discovery)
Target Labels	Target labels are provided during training	No target labels exist in the dataset
Primary Objective	Assign a new sample into a predefined class	Discover organic subgroups based on similarity
Real-world Example	Medical diagnosis: Disease present (Yes / No)	Market segmentation: High, medium, low spenders
Our Lab Algorithms	Decision Tree, k-NN, Naive Bayes	k-Means, Hierarchical Clustering

INSTRUCTOR NOTE

Have students note this table down. In university exams and technical interviews, "Differentiate between Supervised Classification and Unsupervised Clustering" is a guaranteed question.

What is Classification?

Classification means mapping input data features to a **predefined discrete class**.

🎓 Example 1: Student Academic Standing

Inputs (Features): Study Hours + Class Attendance

Target Class: **Pass** or **Fail**

Input: [Study: 6 hrs, Attendance: 85%] → Prediction: **PASS**

✉ Example 2: Email Filtering

Inputs (Features): Subject line, sender address, keywords

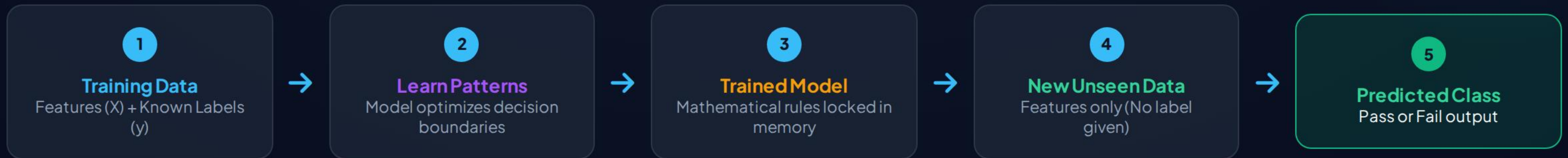
Target Class: **Spam** or **Not Spam (Inbox)**

Input: "Congratulations! Claim prize money" → Prediction: **SPAM**

INSTRUCTOR NOTE

Emphasize the word "Discrete". In classification, outputs are finite categories (Yes/No, Red/Blue, Class A/B). If the output was a continuous number like predicting exact percentage (e.g. 78.4%), that is Regression, not Classification.

How Classification Works



🎓 Training Phase

We feed the model hundreds of past records with both inputs and true outcomes. The algorithm adjusts its internal parameters to minimize error.

🔍 Testing / Prediction Phase

A new student walks into college. We record their study hours and attendance. The model outputs a prediction immediately without human intervention.

INSTRUCTOR NOTE Draw attention to X (features) and y (target). In Scikit-Learn, we will always call `model.fit(X_train, y_train)` for training, and `model.predict(X_new)` for testing.

Decision Tree: Asking Sequential Questions

A **Decision Tree** breaks down complex decisions into a sequence of simple, transparent if-else questions based on feature values.



1. Root Node

The very first question at the top of the tree, representing the most informative feature.

2. Decision Node / Branch

Splits into sub-branches based on the answer (Yes or No).

3. Leaf Node (Terminal)

The final output label (Pass or Fail) where no further questions are asked.

INSTRUCTOR NOTE Highlight interpretability: Decision Trees are popular because unlike black-box deep learning, humans can inspect and explain every single decision branch.

Real-Life Analogy: The Umbrella Decision

Everyday Human Logic

Every morning in coastal Karnataka (Kundapura), you decide whether to carry an umbrella:

- **Question 1:** Is it currently raining outside?
→ If *YES* → **Take Umbrella!**
- **Question 2:** If *NO*, is the sky heavily overcast?
→ If *YES* → **Take Umbrella!**
→ If *NO* → **Leave Umbrella at home.**

Machine Learning Translation

A Machine Learning model follows the exact same structure:

1. **Features:** Humidity, Wind Speed, Cloud Cover
2. **Threshold Test:** Is Humidity > 80%?
3. **Branch Selection:** Traverse left or right
4. **Prediction:** 1 (Carry) or 0 (Don't Carry)

The algorithm finds the best threshold split automatically from past data.

INSTRUCTOR NOTE

Connect this everyday Kundapura monsoon experience directly to data attributes. Tell students: "You are already using decision trees in your brain every single morning!"

Learning from Example Records

Tiny Training Dataset (5 Students)

Student ID	Study Hours (X1)	Attendance % (X2)	Result (y)
S1	2	60%	Fail
S2	3	65%	Fail
S3	4	70%	Pass
S4	5	75%	Pass
S5	6	80%	Pass

🧠 What Does the Tree Learn?

By observing these 5 rows, the Decision Tree algorithm detects a pure split point:

```
IF Study_Hours ≥ 4:  
  → Prediction: PASS  
ELSE:  
  → Prediction: FAIL
```

Notice how 100% of students studying 4+ hours passed. The algorithm creates a clean boundary with zero training error.

INSTRUCTOR NOTE Point out the split boundary at 3.5 or 4 hours. Explain that the algorithm searches through all possible split values to find the one that best separates Pass from Fail.

Decision Tree in Python (Scikit-Learn)

01_CLASSIFICATION/01_DECISION_TREE/DECISION_TREE.PY PYTHON 3.10+

```
# 1. Import the classifier
from sklearn.tree import DecisionTreeClassifier

# 2. Instantiate the model
model = DecisionTreeClassifier()

# 3. Train on data (Features & Labels)
model.fit(X_train, y_train)

# 4. Predict new student: 6 hrs, 80% att
prediction = model.predict([[6, 80]])
print("Predicted Result:", prediction)
```

Step 1 & 2: Import & Initialize

Scikit-learn gives us pre-built, highly optimized decision tree classes.

Step 3: .fit(X, y)

This single line builds the tree branches and calculates information gains internally.

Step 4: .predict()

Pass new feature vector `[[6, 80]]` to receive the predicted label instantly.

📁 See `01_classification/01_decision_tree` in the repository for the full working script.

INSTRUCTOR NOTE Action: Switch screen to VS Code and open ``01_decision_tree/decision_tree.py``. Run it in terminal so students see ``['Pass']`` printed in real time.

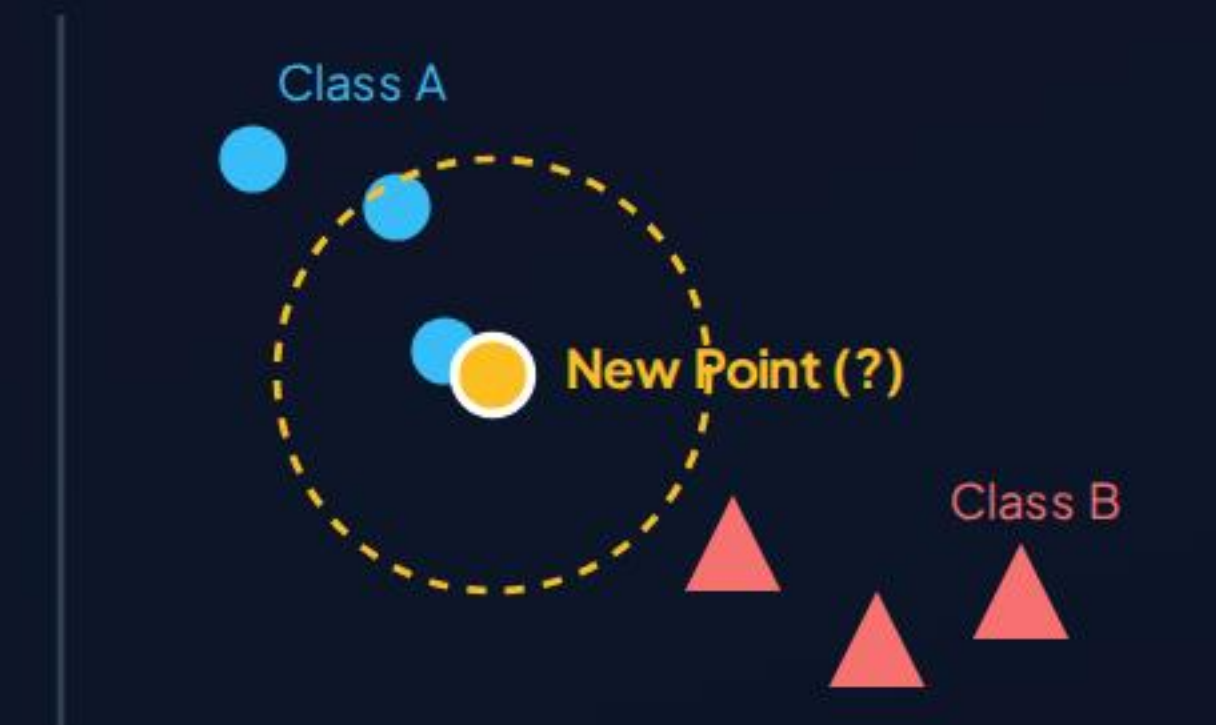
k-Nearest Neighbors (k-NN)

“Tell me who your neighbors are, and I will tell you who you are.”

To classify a new unknown data point, k-NN calculates Euclidean distances to all known points in feature space and inspects the closest **K** neighbors.

Key Philosophy:

Similar things exist in close proximity. If a new student's study hours and attendance resemble three students who passed, that student will most likely pass too.



Yellow dashed circle represents nearest neighborhood radius

INSTRUCTOR NOTE Draw a point on whiteboard between two distinct clusters. Ask students: "If you had to vote based on distance alone, what team does this point belong to?"

How k-NN Works: The Voting Process



Concrete Voting Walkthrough (Setting $K = 3$):

Neighbor 1: Distance = 1.2 → Class A

Neighbor 2: Distance = 1.8 → Class A

Neighbor 3: Distance = 2.4 → Class B

VOTE COUNT

Class A (2) vs Class B (1)

Prediction = Class A Wins!

INSTRUCTOR NOTE

Emphasize why we usually pick odd numbers for K (like 3, 5, 7) for binary classification: it prevents 50–50 tie situations in voting.

What Does K Mean in k-NN?

K = 1

Looks strictly at the single closest point.

⚠ **Risk:** Highly sensitive to noise and outliers. A single mislabeled record ruins prediction.

K = 3 or 5 (Balanced)

Considers a small committee of closest samples.

✅ **Sweet Spot:** Smooths out noisy outliers while capturing local decision patterns.

K = Very Large (e.g. 50)

Considers points far away across the entire dataset.

ⓘ **Risk:** Decision boundaries become too flat; dominant majority class always wins.

General Rule of Thumb: Choose an odd value like $K = 3$ or $K = 5$ to guarantee a clear majority decision in binary classification.

INSTRUCTOR NOTE

Ask students: "If there are 100 students in the class, 90 boys and 10 girls, what happens if we set $K = 99$?" The boys will win every vote! That's why large K over-smoothes.

k-NN Example: Body Health Classification

📁 Known Patient Database

Person	Height (cm)	Weight (kg)	Status
Person A	170	65	Normal
Person B	172	68	Normal
Person C	160	85	Overweight
Person D	175	70	Normal

👤+ Classifying New Patient X

New Patient: Height = 171 cm, Weight = 66 kg

- Step 1:** Distance to Person A → ~1.41 (Very close)
Step 2: Distance to Person B → ~2.23 (Close)
Step 3: Distance to Person D → ~5.65 (Close)
Step 4: Distance to Person C → ~21.9 (Far away)

3 Closest Neighbors: Normal, Normal, Normal → Predicted: NORMAL

INSTRUCTOR NOTE

Demonstrate how intuitive distance is. Point out that Person X is physically adjacent to Persons A, B, and D in height/weight space.

k-NN in Python (Scikit-Learn)

01_CLASSIFICATION/02_KNN/KNN_CLASSIFIER.PY

PYTHON 3.10+

```
# 1. Import KNeighborsClassifier
from sklearn.neighbors import KNeighborsClassifier

# 2. Set K = 3 neighbors
model = KNeighborsClassifier(n_neighbors=3)

# 3. Fit features and labels
model.fit(X_train, y_train)

# 4. Predict new person: 170cm, 65kg
prediction = model.predict([[170, 65]])
print("Predicted Class:", prediction)
```

n_neighbors=3 Parameter

This tells the algorithm to query exactly the 3 closest records when classifying.

Lazy Learning Characteristic

k-NN does not build an explicit mathematical formula during `.fit()`; it simply stores the data and does calculations on-the-fly during `.predict()`!

📁 See `01_classification/02_knn` in our GitHub repo for the complete dataset and execution.

INSTRUCTOR NOTE

Introduce the term "Lazy Learner" in casual terms: k-NN is like a student who doesn't study before exam day, but memorizes the textbook and looks up the closest page when answering questions!

Why Feature Scaling Matters in k-NN

⚠️ The Problem: Scale Imbalance

Imagine classifying loan applicants with two features:

- **Age:** Ranges from 20 to 60 (Diff: ~40)
- **Salary:** Ranges from 20,000 to 2,00,000 (Diff: ~1,80,000)

Because Euclidean distance squares differences:

$$\text{Distance} = \sqrt{(\Delta\text{Age})^2 + (\Delta\text{Salary})^2}$$

Salary completely dwarfs Age! Age becomes invisible.

🛡️ The Solution: StandardScaler

We normalize all features to the same numerical scale (e.g. 0 to 1 or zero mean):

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

After scaling, both Age and Salary contribute equally to distance calculations.

INSTRUCTOR NOTE

Emphasize: "Any algorithm relying on distance metrics (k-NN, k-Means) MUST have scaled features. Decision Trees do not care about scaling because they split one feature at a time."

Naive Bayes: Probability–Based Classification

Naive Bayes uses **Bayes' Theorem of Probability** to decide which class is most likely.

Incoming Message Scenario

"You won a free prize! Click to claim lottery money now."

Question for the Model: What is the probability this is **Spam** vs **Not Spam**?

How It Thinks

The model consults frequency counts from historical training emails:

- When the word "free" appears → 85% of past emails were Spam.
- When "prize" appears → 92% of past emails were Spam.
- When "meeting" appears → 95% of past emails were Not Spam.

By multiplying individual word probabilities together, Spam receives a vastly higher final score!

INSTRUCTOR NOTE

Reassure students: Don't fear Bayes Theorem formula $P(A|B) = P(B|A)P(A)/P(B)$. Focus on the core logic: "Count how often words appeared in spam previously, and multiply probabilities."

Why is it Called “Naive”?

The “Naive” Assumption

It is called **naive** (innocent / simplistic) because it assumes that **all features are completely independent of each other**.

In reality, words occur in coupled contexts:

“Free” and “Gift” often travel together.

“Discount” and “Offer” are related.

Naive Bayes pretends each word acts completely on its own, ignoring grammar and word order!

Why It Still Works Miracles!

Even though the independence assumption is mathematically untrue in real life:

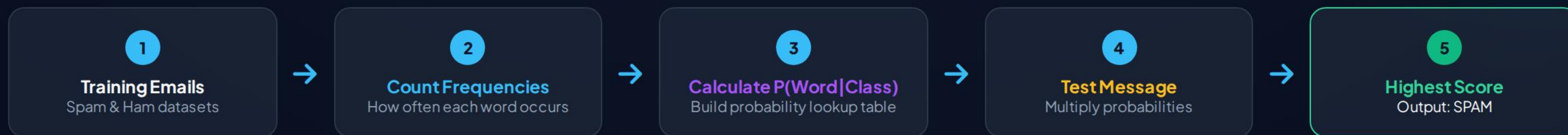
- ✓ Extremely fast training (just counting frequencies)
- ✓ Requires very little memory
- ✓ Unmatched baseline accuracy for text/spam detection

“All models are wrong, but some are useful.” — George Box

INSTRUCTOR NOTE

Emphasize this definition! MCA examiners love asking: “Why is the Naive Bayes classifier called naive?” The single line answer is: “Because it assumes feature independence.”

Naive Bayes Workflow for Text



Simple Probability Comparison

$P(\text{Spam} | \text{"win free cash"})$

0.00084

Higher Probability → WINNER

$P(\text{Not Spam} | \text{"win free cash"})$

0.00002

Lower probability

INSTRUCTOR NOTE Highlight: We don't even need the denominator $P(\text{Data})$ because it is identical for both classes. We just compare the numerators!

Naive Bayes in Python (MultinomialNB)

01_CLASSIFICATION/03_NAIVE_BAYES/SPAM_CLASSIFIER.PY PYTHON 3.10+

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
# 1. Convert text words to frequency counts
vectorizer = CountVectorizer()
X_counts = vectorizer.fit_transform(emails)
```

```
# 2. Train Naive Bayes model
model = MultinomialNB()
model.fit(X_counts, y_labels)
```

```
# 3. Predict new email
new_test = vectorizer.transform(["Claim free prize"])
print("Prediction:", model.predict(new_test))
```

Why CountVectorizer?

Computers don't understand words; they understand numbers. CountVectorizer turns sentences into word frequency matrix columns.

Why MultinomialNB?

Multinomial Naive Bayes is designed specifically for discrete count data like word frequencies in text mining.

 Complete spam filter script ready in 01_classification/03_naive_bayes.

INSTRUCTOR NOTE

Demonstrate the Python script `03\_naive\_bayes/spam\_classifier.py`. Show students how typing "Meeting tomorrow at 10 AM" outputs "Ham", but "Win free cash now" immediately outputs "Spam".

Classification Recap: Quick Comparison

Algorithm	Core Mechanism	Ideal Real-World Use Case
Decision Tree	Sequential If-Else Question Branches	Loan approvals, medical triage where rules must be explainable
k-NN (Nearest Neighbors)	Distance Calculation + Majority Vote	Recommendation systems, biometric matching with continuous metrics
Naive Bayes	Probability Multiplication (Independence)	Spam filtering, sentiment analysis, document classification

💬 Classroom Question:

If you have a dataset with 50,000 text reviews, which algorithm will train faster?

Naive Bayes!

INSTRUCTOR NOTE

Pause here for 2 minutes. Ask 2 students to summarize in their own words what makes Decision Tree different from k-NN. Then transition to Section 2: Clustering.

What is Clustering?

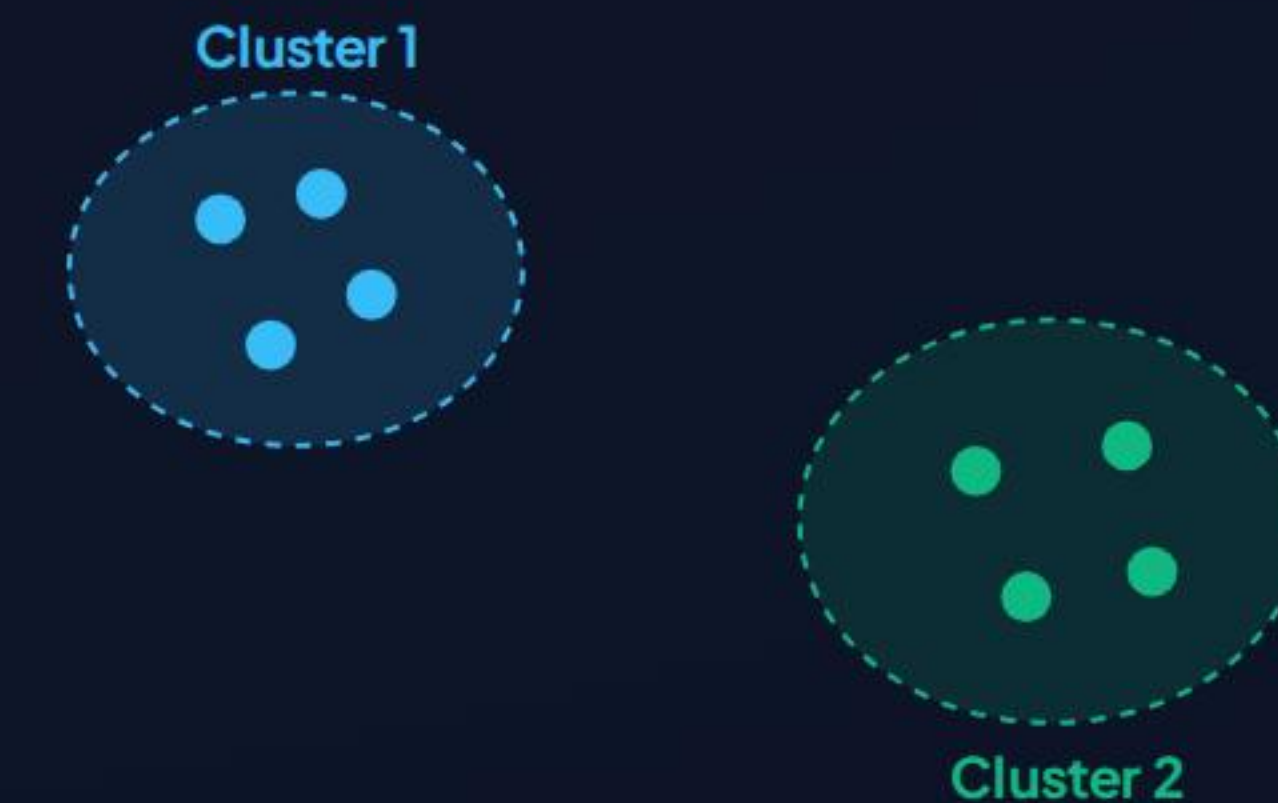
Clustering groups similar data points together **without any prior labels**.

In clustering, we are given raw data with **no predefined target answers**. We ask the machine: *"Find the natural families or clusters that exist within this crowd."*

E-Commerce Customer Example:

Input: 50,000 customers with spending & visit frequency.

Output: Group 1 (Bargain hunters), Group 2 (Occasional luxury buyers), Group 3 (Daily commuters).



High intra-cluster similarity; high inter-cluster separation

INSTRUCTOR NOTE Key phrase to teach: "High intra-cluster similarity, low inter-cluster similarity." Points inside a cluster should be tight friends; different clusters should stay far apart.

Visualizing: Classification vs. Clustering

Classification (Labels Given)

Data + Known Labels Provided

- Every row already marked Red or Blue.
- Goal: Draw a boundary line to separate them.

Question: “Is this newly received email Spam or Not Spam?”

Clustering (No Labels Given)

Raw Data Only (All grey dots)

- No pre-existing classes or color tags.
- Goal: Discover if grey dots naturally form islands.

Question: “Can we discover 3 natural customer groups in our mall data?”

INSTRUCTOR NOTE

Use colors as an analogy: In classification, the dots arrive painted blue and red. In clustering, all dots are plain gray; the algorithm paints them based on who sits near whom.

k-Means Clustering: Partitioning by Centroids

k-Means partitions **N** data points into **K** distinct non-overlapping clusters.

🎯 What is K in k-Means?

K is the exact number of clusters you want the algorithm to create.

- If **K = 2**: Data is split into 2 groups.
- If **K = 3**: Data is split into 3 groups.

You must specify K before the algorithm begins running.

📍 What is a Centroid?

A **centroid** is the imaginary center point (the average/mean position) of all points inside that cluster.

Centroid Coordinates = Average of X and Average of Y for all members belonging to that group.

Every cluster has exactly one centroid.

INSTRUCTOR NOTE Warn students not to confuse "K" in k-NN with "K" in k-Means! In k-NN, K = number of neighbors voting. In k-Means, K = number of clusters being formed.

How k-Means Works: The 4-Step Cycle

Step 1 Choose K

Specify desired number of clusters (e.g., $K = 2$).

Step 2 Random Centroids

Pick K initial centroid points randomly on the plot.

Step 3 Assign Points

Each point attaches to its nearest centroid.

Step 4 Move Centroid

Recalculate centroid to the center of its new members.

REPEAT Steps 3 & 4 until Convergence!

When centroids stop moving and no points change clusters, the algorithm has finished.

INSTRUCTOR NOTE

Analogy: Think of political leaders (centroids). At first, leaders stand anywhere. Voters gather around the closest leader. Then leaders shift their position to the exact center of their supporters!

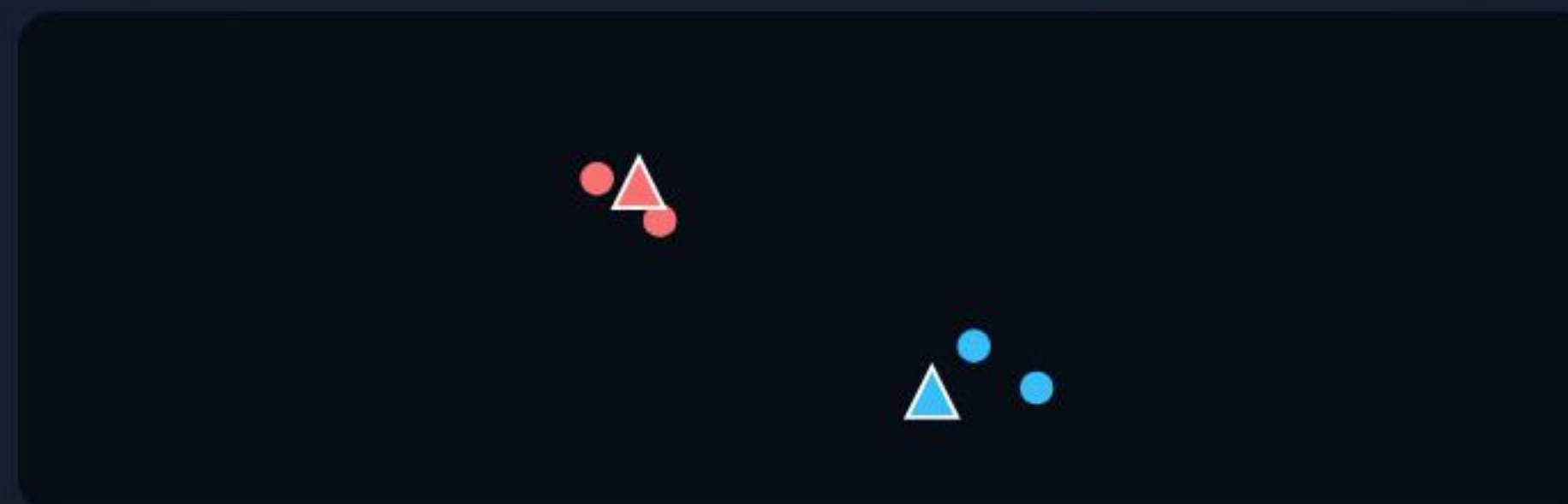
k-Means Visual Convergence

Iteration 0: Random Starts



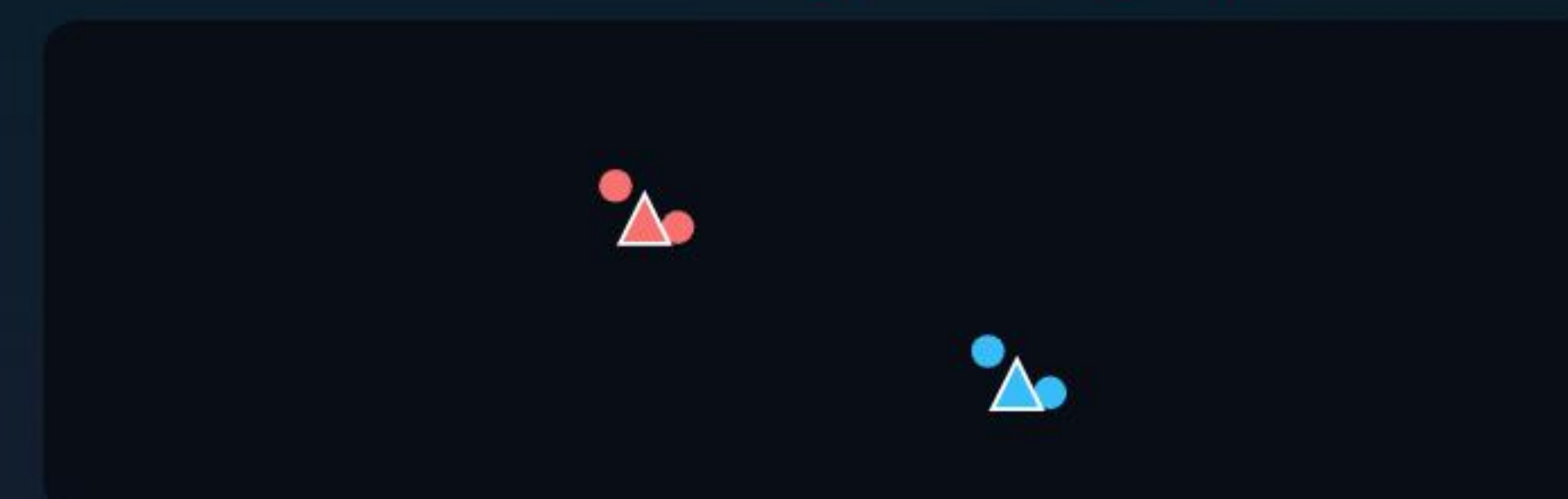
Centroids placed at random coordinates

Iteration 1: Points Assigned



Centroids shift toward cluster centers

Final: Stability (Converged)



Zero change in centroid position

INSTRUCTOR NOTE Point out how fast k-Means converges (often in fewer than 10 iterations for standard classroom datasets).

k-Means in Python (Scikit-Learn)

02_CLUSTERING/01_KMEANS/KMEANS_CLUSTERING.PY

PYTHON 3.10+

```
from sklearn.cluster import KMeans

# 1. Initialize with K = 2 clusters
model = KMeans(n_clusters=2, random_state=42)

# 2. Fit on unlabelled data X (No y required!)
model.fit(X)

# 3. Retrieve results
labels = model.labels_          # [0, 0, 1, 1...]
centers = model.cluster_centers_ # Centroid (X,Y)

print("Assigned Clusters:", labels)
```

Notice: `model.fit(X)`

Look closely: we pass ONLY `X`! In classification we did `.fit(X, y)`. In clustering, there is no `y` label!

The Output Attributes

- `model.labels_`: Cluster tag for each data point.
- `model.cluster_centers_`: Coordinate points of centroids.

📁 See `02_clustering/01_kmeans` for the full Matplotlib scatterplot demo.

INSTRUCTOR NOTE

Demonstrate ``kmeans_clustering.py`` from the repo. Show how Matplotlib plots the data points with different colors for cluster 0 and 1, plus a big red X for the centroids.

Hierarchical Clustering: Bottom-Up Merging

Hierarchical clustering builds a **tree of nested clusters** instead of a single flat partition.

↑ Agglomerative (Bottom-Up)

1. Start with every single data point as its own tiny cluster.
2. Find the two closest points and **merge** them.
3. Repeat merging closest groups until all points become one single super-cluster.

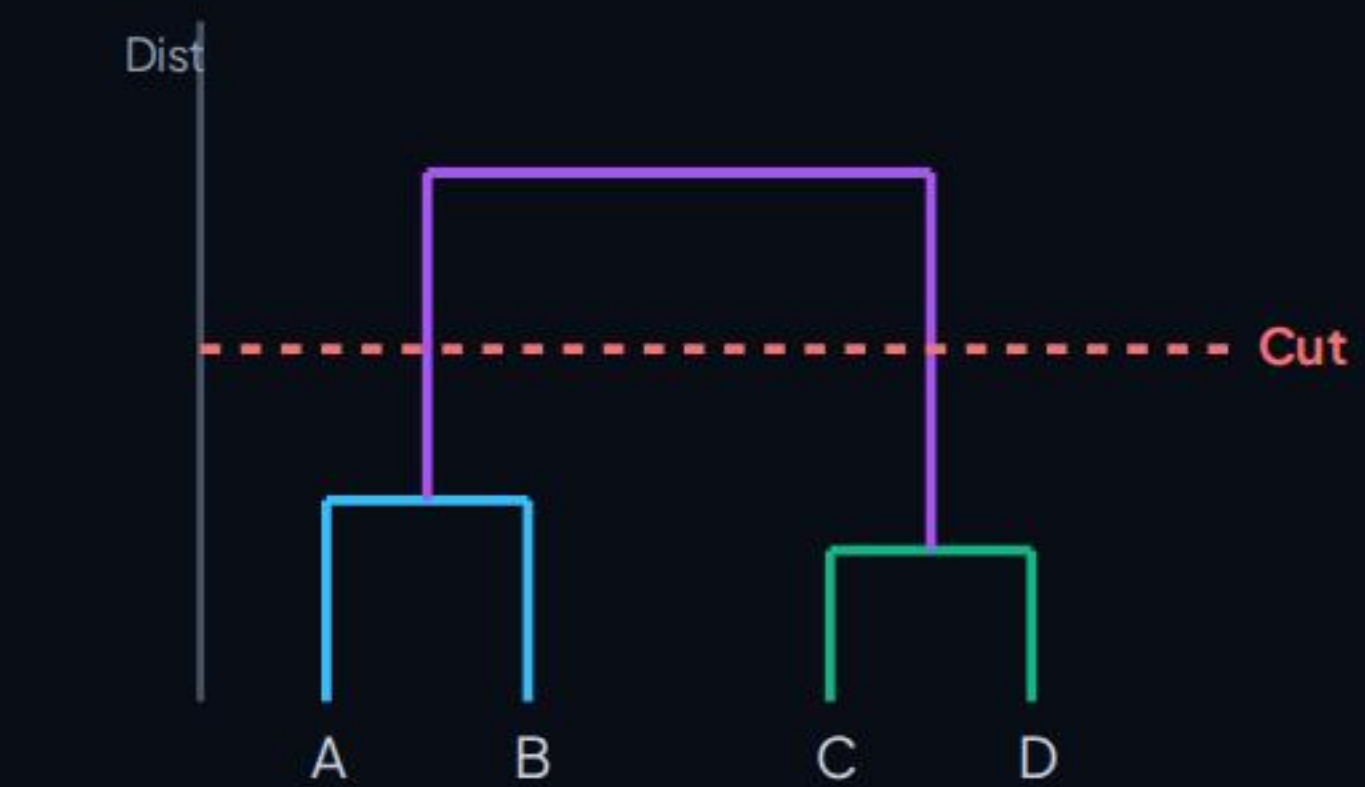
The Merging Progression:

```
Start: [A] [B] [C] [D] [E]
↓ Merge closest: (A+B) (C+D) [E]
↓ Merge next closest: (A+B+E) (C+D)
Final: (A + B + C + D + E) All in 1 Root
```

INSTRUCTOR NOTE

Classroom analogy: Imagine students entering an auditorium as strangers. First, desk-mates become pairs. Then pairs join neighboring benches. Finally, the whole class is one batch!

Understanding a Dendrogram



Cutting horizontal red line yields exactly 2 clusters

1. Leaves (Bottom)

Individual data points (A, B, C, D) before any merging begins.

2. Vertical Height

Represents the Euclidean distance between clusters when they merged. Tall lines mean merged clusters were far apart.

3. Cutting the Dendrogram

Draw a horizontal line across. The number of vertical lines it slices through determines your number of clusters!

INSTRUCTOR NOTE

Show how a dendrogram solves the "How do I choose K?" dilemma. By looking at the longest vertical lines without crossings, we identify the most natural cluster count.

Hierarchical Clustering in Python

02_CLUSTERING/02_HIERARCHICAL/HIERARCHICAL.PY

PYTHON 3.10+

```
from sklearn.cluster import AgglomerativeClustering
import scipy.cluster.hierarchy as sch

# 1. Generate Dendrogram with SciPy
dendrogram = sch.dendrogram(sch.linkage(X, method='ward'))

# 2. Fit Agglomerative model with K = 2
model = AgglomerativeClustering(n_clusters=2)
labels = model.fit_predict(X)

print("Assigned Cluster Labels:", labels)
```

Scipy vs. Scikit-Learn Roles

- **SciPy**: Perfect for computing linkages and plotting the visual dendrogram tree.
- **Scikit-Learn**: Provides **AgglomerativeClustering** to cut and output label arrays for every point.

Ward's Linkage Method

The default linkage method **'ward'** minimizes variance within clusters, producing neat, compact groups.

📁 See `02_clustering/02_hierarchical_clustering` in our repository.

INSTRUCTOR NOTE Open `hierarchical.py` in VS Code. Run it so the Matplotlib window pops up with the full color-coded dendrogram chart.

k-Means vs. Hierarchical Clustering

Feature	 k-Means	 Hierarchical
Specification of K	Must specify K strictly beforehand	Does not require K initially (dendrogram decides)
Centroids	Relies heavily on centroids and means	Does not use centroids; merges by pair distances
Visual Output	Partitioned scatter points	Full hierarchical Dendrogram tree
Dataset Scalability	Very fast: Works on millions of records	Computationally heavy: Slow on massive datasets
When to Choose	Large, flat e-commerce / user datasets	Taxonomies, biology, small nested datasets

INSTRUCTOR NOTE

Summarize: If you have 1,000,000 records, choose k-Means because hierarchical clustering pairwise distance matrix will run out of RAM!

Complete Mining Techniques Map

MINING TECHNIQUES

CLASSIFICATION (Supervised)

Decision Tree
“Questions”

k-NN
“Neighbors”

Naive Bayes
“Probability”

CLUSTERING (Unsupervised)

k-Means
“Centroids”

Hierarchical
“Merging”

INSTRUCTOR NOTE

Have students recite the 5 one-word summaries: Decision Tree → Questions; k-NN → Neighbors; Naive Bayes → Probability; k-Means → Centroids; Hierarchical → Merging.

From Theory to Python Implementation



Our Repository Architecture

```
01_classification/  
├── 01_decision_tree/  
├── 02_knn/  
└── 03_naive_bayes/
```

```
02_clustering/  
├── 01_kmeans/  
└── 02_hierarchical_clustering/
```

INSTRUCTOR NOTE

Instruct all students to clone or pull the repository onto their lab machines. Every folder has a clean, standalone script that matches today's slide numbers.

Quick 5–Minute Classroom Quiz

1. Is Classification supervised or unsupervised?

Hint: Do we give the true answers during training?

2. In k–NN, what does K stand for?

Hint: Think about who votes on the new point.

3. Why is Naive Bayes called “Naive”?

Hint: What bold assumption does it make about features?

4. What is a centroid in k–Means?

Hint: What geometric property does it represent for its cluster members?

5. What does a Dendrogram represent in Hierarchical Clustering?

Hint: How does it visualize distances and cluster merges?

INSTRUCTOR NOTE

Engage the classroom! Call on individual students around the room for each question. Answers: 1. Supervised; 2. Number of nearest neighbors; 3. Feature independence; 4. Mean center point of cluster; 5. Tree showing cluster merge hierarchy.

Key Takeaways & Next Steps

Classification Summary

- **Decision Tree:** If-Else questions
- **k-NN:** Look at closest neighbors
- **Naive Bayes:** Calculate probabilities

Clustering Summary

- **k-Means:** Discover centroids iteratively
- **Hierarchical:** Build bottom-up merge tree
- **Unsupervised:** Zero labels required

**“Understand the core intuition first.
Then let Python do the heavy lifting.”**

MITK • Department of Computer Applications • M.Tech CSE

INSTRUCTOR NOTE

Thank the students for their attention. Direct them to start the hands-on lab exercise in their machines.